

pi 解剖：四个工具， 六个"不做"， 和一个 9.9 万星的实验

A MINIMAL AGENT HARNESS

4

TOOLS

6

NO'S

98.8k

STARS*

250+

RELEASES

* 单源页面抓取（2026-08-29），未能二次核验。全文基于公开材料写成，引语保留英文原句，单一看源的数据当场标注；没跑过基准，没有利益相关。

2025-11-30 发布



2026-01-31 Ronacher 评述



2026-04-08 加入 Earendil



2026-08-28 v0.84.4

前半回答"它是什么、凭什么", 后半回答"代价是什么"

01 缘起: 为什么又是一个 agent P3

02 架构解剖: 走查 / 时序 / 五层 / 数字 P4-7

03 四工具哲学 + 六个"不做" P8-9

04 上手: 十分钟到会话树 P10-11

05 扩展系统 + 完整源码 P12-13

06 横向定位: 底盘 vs 整车 P14-15

07 批判: 六笔账 + 三段原声 + 边界 P16-18

REF 提炼 + 参考来源 P19-20

METHOD 全文基于公开材料 (README、源码、作者博客、第三方批评), 每处引用给出处; 没跑过基准, 没有利益相关。

01 缘起：为什么又是一个 agent

WHY

动机

反塞满 · 反黑盒

"往上下文窗口里塞了太多东西，还把正在发生的事藏起来"——两个不满，两个设计决策

写 pi 的人叫 Mario Zechner，GitHub ID 是 badlogic——Java 游戏框架 libGDX 就是他的作品，社区养了十几年还活着。这种履历自带一种信用：他见过一个项目怎么长大，也见过它怎么死。

按他自己的说法，做 pi 是因为现有的 coding agent "往上下文窗口里塞了太多东西，还把正在发生的事藏起来"（大意，出自其 2025-11-30 的设计博文，原文为英文）。这话有两层不满：前一层是空间上的，系统提示词、工具说明、各种隐藏注入，还没开始干活，上下文先被吃掉一大截；后一层是黑盒，工具在后台做了什么，你只能看个大概。pi 把这两个不满都做成了设计决策，后面每一章都能看到回声。

pi.dev 官网把一句话挂在首页，算是定位的自白："There are many agent harnesses but this one is yours"。harness 指包在模型外面、负责跑工具管状态的那层程序，直译过来就是：执行框架有很多，但这一个是你的。所有权，而不是体验，才是 pi 的卖点。

引爆点在 OpenClaw（2026 年初走红，旁证称病毒式传播，未量化核实）——它的底层 agent 框架就是 pi，官网把 OpenClaw 列为 SDK 集成实例。做底盘的项目，因为别人在它上面造出了爆款整车，被更多人看见了。第一眼看到 98.8k 的星数（单源抓取，2026-08-29）我以为是写错了，后来想明白，它早就不是"个人项目"三个字能概括的东西。

时间线

2025-11-30 博文发布 → 2026-01-31 Ronacher 评述 → 2026-04-08 《I've sold out》加入 Earendil（仓库与 npm 包名随之迁移）→ 2026-08-28 v0.84.4（8 月 5 连发，累计约 250 个 release）

02 架构解剖：一轮对话是一圈循环

ARCH

产出

agent loop

agent loop 就住在 pi-ai 这层——没有最大步数限制，什么时候算干完，模型自己说了算

你在终端里敲下“帮我修复这个报错”，消息进了 pi-ai，也就是最底下那层。这层名字像模型适配库，实际上把 agent loop 也做进去了：它把你的消息和系统提示词发给模型，模型流式地回，返回里如果带着工具调用，pi-ai 负责把流式碎片里的参数一点点聚合成完整对象，然后执行工具，再把结果作为一条工具消息回填进上下文，紧接着再问一次模型。循环，直到模型不再发起工具调用为止。

这层还承包了所有和“换个模型”有关的脏活：流式输出、带 TypeBox schema 校验的工具调用、thinking 支持、跨 provider 的上下文接力、token 和成本统计。跨 provider 接力值得单独说一句——你上半场用 Claude 读完了代码库，下半场可以切到 DeepSeek 继续干活，上下文能跟着走。

中间层 pi-agent-core 在博客里的自我定位是 “a thin, transparent layer”，薄薄一层运行时：Agent 类、状态管理、事件订阅、消息排队，外加 HTTP、WebSocket、内存三种传输抽象。设计口号是 “Everything is an event”，一切皆事件，工具开始执行是一个事件，模型吐了一个 token 也是一个事件。上层的一切界面和扩展，都建立在这个事件流上。层薄还有个常被低估的好处，出问题时你能找到责任在哪层：模型抽风、循环卡死、界面错乱，在 pi 里对应三个包的事，在一体化的工具里是一团黑盒。

上下文纪律同样干脆：整个项目里只有 AGENTS.md 一个东西会被注入提示词（全局一份、项目一份，塞在系统提示词底部），除此之外不加戏，你甚至可以用 .pi/SYSTEM.md 把整个系统提示词换掉。

< 1000 tokens

系统提示词 + 全部工具定义的总和——省下来的上下文，全留给你的代码

把上一页的走查画成图：四方协作，虚线是回程



NOTE 改一个错字一圈收工，跨文件重构十几圈不稀奇——圈数没有上限；headless 模式走 JSON 流或 RPC，可以把 pi 当组件嵌进自己的程序。

02 五层薄切：每层都薄得能一眼看完

ARCH

产出

pi-mono × 5 包

TypeScript 单仓，五个能独立安装的 npm 包，没有一个名字带 platform

pi-coding-agent

CLI 组装层

会话 · 扩展 · 主题 · 上下文文件

三种模式：交互 TUI / print 单发 / headless (JSON 流 + RPC, 可当组件嵌进自己的程序)

↓ 事件流

pi-agent-core

Agent 运行时（薄层）

Agent 类 · 状态 · 事件订阅 · 消息排队 · 传输抽象

↓ 统一调用

pi-ai

统一多 provider LLM API + agent loop

Anthropic / OpenAI / Google / xAI / Groq / Cerebras / OpenRouter + GLM / Qwen / Kimi / DeepSeek / MiniMax, 任意 OpenAI 兼容端点

pi-tui

自研终端 UI: retained mode + 差分渲染, 没用 ncurses

pi-telemetry

厂商中立遥测契约; 博客时代还没有, 后来长出来的

02 十个数字，勾出一个项目的轮廓

BY THE
NUMBERS

抓取

2026-08-29

节奏像呼吸一样稳定：月均 5 个 release，两年发版 250 次

98.8k*

GITHUB STARS

250+

RELEASES 累计

v0.84.4

最新版 · 8 月 5
连发

5

NPM 包 · 单仓五层

<1000

启动 TOKENS

24

斜杠命令

50+

官方扩展示例

~30

API KEY
PROVIDERS

6

刻意不做 · 每个都有替代路径

3

运行模式 · TUI / print / headless

WEAK * 星数为单源页面抓取（2026-08-29），未二次核验；「50+」「~30」为官网与 npm registry 口径。订阅登录另认 Claude Pro/Max、ChatGPT Codex、GitHub Copilot 三家。

四工具哲学：模

03 模糊匹配是 edit 的机关

PHILOSOPHY

产出

read/write/edit/bash

每个工具朴素到近乎无趣，但各有讲究

read 带分页和行号，大文件按 offset 翻页读，输出格式各家模型都被训练过处理，连图片也能读；write 整文件覆盖，没有花活；edit 用最简单的 diff 格式配模糊匹配——模型输出的空格缩进经常有细微漂移，严格匹配会让 edit 频繁失败，容忍几个字符的出入，成功率就上来了，这是个典型的小设计解决大问题的例子；bash 就是 bash，执行命令加超时，长驻进程交给 tmux。

作者的理由写在博客里：模型都被训练过适配这种最小 schema，其他工具多半是 bash 的语法糖，再堆只是浪费上下文。grep、find、ls 这几个只读工具存在，默认关着，pi --tools 想开就开。

系统提示词开头一句是 "You are an expert AI agent connected to a shell"，中间放手让它自主干到完，结尾防幻觉——三段都是设计声明：极简不是没有态度，是把态度写进最少的字里。

”

You don't need to ask for permission. The user expects you to work autonomously until the task is complete. —— 遇到不认识的符号、函数、API，先列出所有候选来源，挨个查证，把理解锚在可验证的代码或文档上。

SYSTEM PROMPT · 模型自己的判断力是第一道防线

grep / find / ls

只读工具存在，默认关着，pi --tools 想开就开。

怎么读系统提示词

开头宣告"专家 agent 连着 shell"，中间放手自主干到完，结尾防幻觉——三段都是设计声明。

03 六个"No"是立场，不是缺口

PHILOSOPHY

产出

fig-3

安全不靠弹窗：靠容器、靠裁剪能力、靠供应链纪律

安全思路一脉相承：无内置权限系统，按启动用户权限直接跑；官方建议放容器（Docker / Gondolin / OpenShell），轻量玩法是只读模式 `--tools read,grep,find,ls`，只带眼睛不带手；供应链纪律也有——`--ignore-scripts`、依赖锁版、最小存活期、脚本白名单。风险没有消失，只是挪到了环境侧。

刻意不做	官方给的替代路径（README Philosophy 原文意译）
No MCP	用带 README 的 CLI 工具（见 Skills），或写扩展加上 MCP 支持
No sub-agents	用 tmux 起 pi 实例，或写扩展，或装别人做好的包
No permission popups	放容器里跑，或用扩展写自己的确认流程
No plan mode	计划写成文件，或写扩展，或装包
No built-in to-dos	原话是“它们会把模型搞糊涂”，用 TODO.md 或扩展
No background bash	用 tmux，全量可观测，直接交互

NOTE 每个 No 后面那句“你可以自己造”才是正主——这六个 No 是立场，不是缺口，第五章看官方交的作业。

04 上手：十分钟跑起来， 坑在 Node 版本

HANDS-
ON

产
出

quick start

双轨登录：API key 按量付费走模型厂，/login 订阅走包月，pi 不抽成

```
quick start

# 安装 (Node ≥ 22.19.0, v0.84.4)
npm install -g --ignore-scripts @earendil-works/pi-coding-agent

# 登录二选一
export ANTHROPIC_API_KEY=sk-ant- ...
pi
# 或交互里走订阅
/login

# 会话操作
pi -c          # 继续最近会话
pi -r          # 浏览历史会话
pi --fork id   # 分叉新会话
```

第一个坑在 Node 版本：engines 字段写死 $\geq 22.19.0$ ，不少机器默认 LTS 还停在 20，装之前先 `node -v`。Windows 不用绕路：官方 `install.ps1`、内置 powershell 工具、Termux 文档都有。

登录是双轨制：一轨 API key，约 30 家 provider 按量付费，钱花在模型厂；另一轨 /login 走订阅，认 Claude Pro/Max、ChatGPT Codex、GitHub Copilot 三家，钱花在订阅里。pi 不在这中间抽成，pi.dev 上至今没有定价页。想跑本地模型，llama.cpp 的 router server 是一等公民（/login llama.cpp 配置、/llama 管下载加载），Ollama 也在列表里。两轨怎么选，算笔小账就清楚：API 贵时一晚烧几十块很正常，但模型随便挑；订阅额度内随使用，跨过额度就等重置。pi 对两轨一视同仁，功能零差异。

04 会话是一棵树：跳分支不改历史，只换路径

HANDS-
ON

产出

/tree

Esc×2 进树视图是最有辨识度的操作；Enter 与 Alt+Enter 是两种排队语义

每个会话是棵树，存在 `~/.pi/agent/sessions/` 按项目目录归档，文件格式是 JSONL，每一行一条记录，记录之间靠 `id` 和 `parentId` 挂成树，你当前所在的位置叫 `active leaf`。用户消息、助手回复、工具调用、压缩摘要，全是树上的节点。跳分支不改历史，只换路径，被放弃的分支还能让 `pi` 自动做个摘要挂在岔路口，回来的时候读一眼就能接上。

这带来一个别的工具没有的自由度：连按两次 `Escape` 进树视图，跳回任意历史节点接着聊。`/fork` 从某条历史消息开新会话，一个是“回到过去改个决定”；`/clone` 把当前分支整个复制走，是“平行世界再来一遍”。打字也有讲究：`Enter` 排进去的是 `steering` 消息，当前这轮工具跑完就插进去，用来纠偏；`Alt+Enter` 排的是 `follow-up`，等它全部干完才送，用来排队喂任务。编辑器里 `@` 可以模糊搜文件，`!command` 直接跑命令把输出喂给模型。

从 Claude Code 迁移过来的用户，肌肉记忆大体能用：`AGENTS.md` 和 `CLAUDE.md` 它都认，`/compact` 它也有，24 个斜杠命令里一半以上似曾相识。最大的适应成本反而是“少”——没有内置待办、没有权限弹窗、没有后台任务面板，刚开始有点空落落的，习惯之后你会发现屏幕上少了很多可点的东西，多了一条完整的命令流。

KEYS `Ctrl+L` 换模型 · `Ctrl+P` 轮换收藏 · `Shift+Tab` 思考档 · `Ctrl+O` 折叠工具输出 ·
`/session` 看 token 与花费 · `/hotkeys` 查全量

05 扩展系统：极简的反面是可编程

EXTENSIONS

产出

ExtensionAPI

六个"No"里被拒掉的能力，在扩展系统里全能造回来——API就三件事

扩展就是一个默认导出的工厂函数。API 就三件事：`pi.registerTool` 注册工具，`pi.registerCommand` 注册命令，`pi.on` 挂事件，事件里最常用的是 `tool_call`，能拦截每一次工具调用。扩展放在 `~/.pi/agent/extensions/`（全局）或 `.pi/extensions/`（项目），改完 `/reload` 立即生效。打包分发也成体系：`package.json` 里声明 `pi` 字段，发到 npm 打上 `pi-package` 关键词，或直接给 git 地址，别人 `pi install` 一条命令装好。官方文档有个不加修饰的提醒：`pi` 包以你的完整系统权限运行，装第三方包之前先读源码。

`skills` 那套遵循 `agentskills.io` 标准，`markdown` 文件写清楚“什么时候用我、步骤是什么”，`/skill:name` 调用；`prompt template` 放个 `markdown` 文件就是 /模板名，支持 `{{变量}}` 占位。官方仓库躺着 50 多个扩展示例——六个“No”每个都有参考实现：`subagent/`、`plan-mode/`、`permission-gate`、`sandbox/`，甚至有等结果时打 `Doom` 的扩展。社区也已有成型第三方包：`oh-my-pi` 把 `hash` 锚定编辑、`LSP`、浏览器、子代理打包进来，`pi-mom` 把 `Slack` 消息委派给 `agent`（口径来自官网与二手综述，未逐仓核验）。作者本人更绝，把自己的开发会话全部公开在 `Hugging Face` 上，任何人可以下载真实工作记录，看这个项目是怎么被它自己造出来的。

当然，“能自己造”和“已经造好”是两回事——官方示例到生产可用之间那段路，得你自己走。这笔账，下一章算。

05 一个扩展的完整源码

EXTENSIONS

产出

protected-paths.ts

拦截 `tool_call` 事件，命中保护路径就返回 `block`——半分钟能读完的全部行为

```
packages/coding-agent/examples/extensions/protected-paths.ts
import type { ExtensionAPI } from "@earendil-works/pi-coding-agent";

export default function (pi: ExtensionAPI) {
  const protectedPaths = [".env", ".git/", "node_modules/"];

  pi.on("tool_call", async (event, ctx) => {
    if (event.toolName !== "write" && event.toolName !== "edit") {
      return undefined;
    }
    const path = event.input.path as string;
    const isProtected = protectedPaths.some((p) => path.includes(p));
    if (isProtected) {
      if (ctx.hasUI) {
        ctx.ui.notify(`Blocked write to protected path: ${path}`, "warning");
      }
      return { block: true, reason: `Path "${path}" is protected` };
    }
    return undefined;
  });
}
```

半分钟读完

这才是重点：你真的可以读完你的 agent 的全部行为。

社区已经造好的

oh-my-pi (hash 锚定编辑+LSP+浏览器+子代理)、pi-mom (Slack 委派)、@termdraw/pi (画图)、pi-doom (等结果打 Doom)。

06 横向定位：分歧点只有一个

能力边界由谁决定，是 pi 和另外两家的分水岭

维度	pi	Claude Code	OpenCode
定位	极简 agent harness	官方全能 coding 工具	开源多形态 agent
许可	MIT，全开源	闭源商业产品（订阅制；业内通识，官方页未标注，存疑）	开源（维护方 Anomaly）
内置工具	4 个核心 + 可选只读	面宽（未公开枚举）	未枚举
MCP	刻意不做，可扩展补	有	有
sub-agent	无内置，有示例扩展	有（lead agent 协调）	有 Agents 形态
扩展模型	TS 扩展 + pi package	MCP + hooks + skills	MCP + plugins + SDK
模型	16+ 供应商 + 本地 llama.cpp，会话内热切换	Anthropic 为主，可第三方 key	多供应商（另有 OpenCode Zen 策展列表）
形态	纯终端（TUI/print/headless）	终端 + 桌面 + IDE 插件	终端 + 桌面 + IDE
计费	免费，pi.dev 无定价页	订阅制	未标注

空着的格子是未取得一手数据的地方，宁可空着不编。

06 选底盘还是选整车，取决于你想不想要所有权

COMPARE

补充

模型自由度

没有对错，只有你想要什么

底盘：pi

每一处都能改：扩展、skills、主题全是你的代码；半分钟读完一个扩展的全部行为。

代价：组装和维护归你——大仓检索靠 grep 加耐心，并行靠 tmux 自排，自建扩展随模型升级自己修。

整车：Claude Code / OpenCode

上路就能开：MCP、sub-agent、权限确认、plan mode、后台任务全是产品功能，IDE 插件和桌面端齐备。

代价：路线是别人画的，能力边界由厂商决定，模型选择受订阅框架约束。

BONUS pi 的 /model 会话内热切换 + 跨 provider 上下文接力：上半场 Claude 读库、下半场 DeepSeek 干活，一条 workflow 混用两家模型是常规操作。OpenCode 走策展思路（OpenCode Zen 精选列表），pi 走全量开放，接什么模型你自己定。

极简不是免费的，六笔账每笔都有出处

第一笔，无 MCP。连 pi 的头号拥趸都把这条列为第一缺点。Flask 作者 Ronacher 在那篇评述里写："The most obvious omission is support for MCP. There is no MCP support in it." 这里必须交代立场：那篇文章整体是软文，Ronacher 自称已经成了 shill（推销客），而且他和 Zechner 在同一家公司。自己人说"这缺失最明显"，分量反而更重。MCP 在 2026 年已是事实标准，不兼容意味着所有现成 MCP 服务器都不能开箱即用。官方的替代路线是写扩展或桥接，而 Ronacher 对桥接可靠性的原话是："Or not, I don't know :)"——拥趸对自家核心替代路径的态度是"我不知道"，这句比任何批评都诚实。

第二笔，无现成生态。同一篇文章里还有一句更狠的："There is no MCP, there are no community skills, nothing."（2026 年 1 月的判断。）Reddit 上有篇实测帖，说法类似："No find tool, no git tool. No sub-agents built in."（转引自搜索摘要，原帖被反爬拦住，我没读到楼主最终结论。）把三句放在一起读，能力获取基本等于自己写代码。公平地说，第五章列的社区包是存在的，只是它们和 marketplace 式即插即用生态不是一个量级。

第三笔，TS 门槛，以及它背后的账。官方对"不满意"的标准答案是直接重写 internals，它就是一堆可替换的 TypeScript 文件。这话的另一面是，深度定制的入场券是会写 TypeScript；而且自建的扩展是你自己的代码，模型升级行为漂移了，得你自己修——自己写扩展省下的配置成本，会变成维护一堆自己生成的 TS 代码的成本。要说明白，我没搜到非 TS 用户抱怨这一点的直接引语，这是我的推断，不是用户的声音。

后三笔和钱、人、规模有关

第四笔，商业化悬念。2026 年 4 月加入 Earendil 的消息出来后不久，r/LocalLLaMA 上就有人立帖，标题是 "pi.dev enroute to enshitification?"。Zechner 在官宣博文里的承诺原文是："pi is MIT licensed. It will stay MIT licensed. You can use it, fork it, build products on top of it, sell those products. Nothing changes." 同时，未来的商业功能会走 Fair Source 许可（一种源码公开但限制竞品直接使用的许可思路，来自第三方对 RFC 0015 的转述）。到发稿为止，我没找到开源用户被背刺的实例，风险是结构性的，不是已然。但有一条成本已经真实发生：仓库从 badlogic 迁到 earendil-works，npm 包名跟着换，装着旧包的用户得自己迁移。

第五笔，好评的圈层。我核引文作者的时候发现，pi 最重要的几篇好评，作者要么在 Earendil，要么和它互相依存，OpenClaw 的作者 Peter Steinberger 也在这个朋友圈里。还有个花絮：HN 上流传最广的那句 "linear flow 真让人沮丧"，我查了原文，是 Zechner 在批评别家，根本不是在骂 pi。这个圈子的好评要打折听——不是因为他们说谎，而是因为圈子太小，好评难免互相捧场，参考价值得自己折算。

第六笔，任务规模的边界。四工具加单人循环的架构，在单人单仓的日常开发里游刃有余，但大型仓库的全库检索、多任务并行的长流程，官方的答案都是 tmux 和扩展。具体一点说，让 pi 在上百个目录的仓库里找一个变量的所有引用，它能靠 grep 加耐心做到，就是慢；同时开三个任务互不干扰地并行，编排、汇总、冲突处理都归你。Claude Code 把这些做成了产品功能，pi 把它们留成了练习题。这不是缺陷，是边界，但买底盘的人应该提前知道自己要开什么路。

写到这里我得承认一件没查明白的事：Earendil 靠什么把 pi 变成生意。pi.dev 上没有定价页，RFC 站点上也读不出商业路线，这是我能确认的全部。

07 五种情况别用 pi, 一种情况闭眼入

CRITIQUE

产出

fig-7

反方视角的适用边界，每一条都有出处

依赖 MCP 服务器生态

接工具要写 TS 扩展或桥接，而桥接可靠性连拥趸都承认"我不知道"。(Ronacher 原话)

要开箱即用的全家桶

sub-agent、内置 git、find、社区 skills 都没有，且这是设计立场不是功能路线图。(Reddit 实测 + "nothing")

不会 TS 又想深度定制

定制的入场券是会写 TypeScript; "重写 internals" 的另一面就是门槛。(推断，非用户引语)

IDE / GUI 重度用户

纯终端 harness，无官方图形前端；线性对话流是刻意选定的交互。

对许可分层过敏的组织

核心 MIT 承诺在，但 Fair Source 商业层已写入路线，预期分化。(RFC 0015 转述)

反过来，谁适合

想完全掌控 harness、愿意读源码、用 TypeScript 的高级开发者与 agent 构建者。

HONEST Earendil 靠什么把 pi 变成生意，我没查明白。pi.dev 上没有定价页，这是我能确认的全部。

提炼：pi 的价值不在"更好用"，在"可拥有"

SUMMARY

pi · A MINIMAL
AGENT HARNESS

你能读完它的全部行为，改掉其中任何一处，然后真正拥有你的 agent

01	缘起是什么 对"塞满上下文、黑盒操作"的集体路线投反对票，顺手被 OpenClaw 点爆	L26/L21
02	架构巧在哪 五层薄切，agent loop 住在模型接入层里，启动 <1000 tokens，一切皆事件	L1/L2/L7
03	四工具的底气 覆盖 coding 全部动作原语，六个"No"各有替代路径，安全挪到环境侧	L3/L4/L5
04	上手难不难 一条 npm 命令，双轨登录，会话树和消息排队值得练熟	L10-L12
05	扩展的真相 "没有的"官方示例全能找到造法，但示例和产品之间隔着你自己写的代码	L13-L15
06	账单有哪些 无 MCP、无生态、TS 门槛、自建维护、商业化悬念、好评圈层	L22-L24

想开现成的车，另外两家更合适；想拥有一台完全按自己想法改装的，pi 值得这场折腾。完整九维对照与全部引文上下文见第 06-07 章。

REF 参考来源：每一条都能 点开验证

SOURCES

抓取

2026-08-29

weak 来源已在正文标注口径

pi 官网与文档 — pi.dev

仓库与 README (Philosophy / Quick Start / Extensions / Sessions) — github.com/earendil-works/pi

npm 包元数据 (engines / 版本) — registry.npmjs.org/@earendil-works/pi-coding-agent/latest

Zechner 设计博文 (2025-11-30) — mariozechner.at/posts/2025-11-30-pi-coding-agent

Zechner 《I've sold out》(2026-04-08) — mariozechner.at/posts/2026-04-08-ive-sold-out

Ronacher 评述 (2026-01-31, 立场见正文) — lucumr.pocoo.org/2026/1/31/pi/

protected-paths.ts 扩展源码 —

[…/examples/extensions/protected-paths.ts](#)

会话文档 — […/packages/coding-agent/docs/sessions.md](#)

GitHub Releases (v0.84.4) — github.com/earendil-works/pi/releases

Reddit 实测评 (转引) — r/AI_Agent_Reviews/comments/1t62nnj

[r/LocalLLaMA 商业化质疑帖 \(转引\)](#) — [r/LocalLLaMA/comments/1u6a499](#)

对比数据 —

code.claude.com/docs/en/overview · opencode.ai/docs

WEAK 星数 98.8k 单源未二次核验；社区包描述未逐仓核验；Claude Code 闭源为业内通识已按存疑处理。

PRINT 打印提示：封面、总结页与各代码卡为深色底，整页打印墨耗约为文字页 20-30 倍；大批量打印可向我要浅色变体。